Sampling and Per-Precision Histograms with nixnan: An Experiment on `rd_nixnan.cu`

ganesh@cs.utah.edu
(written with Claude Code)

September 23, 2026

Abstract

This report documents a self-contained experiment (`Expts-9-23/`) demonstrating that `nixnan.so` helps understand a real GPU precision-sensitivity problem in several specific ways: (1) it can trade instrumentation overhead for observation completeness via the `SAMPLING` env var, (2) that trade-off can be driven through an arbitrary, externally-scripted schedule (here, an exponentially decaying rate) rather than a single fixed value, (3) it reports, per floating-point format, exactly how much of that format's dynamic range a given run actually used, and (4) it can confirm or refute a hypothesis about *why* a lower-precision format fails first — here, that domain enlargement does not explain FP16's earlier failure, but running longer does. All numbers below come from real runs captured in `Logs/`; nothing is simulated or hand-typed. Section 9 covers a bug found along the way in the program's own non-finite check (not in nixnan), the fix, and a run showing FP64 overflowing for real.

Contents

1 The Example **2**

2 "Straight nixnan": How These Runs Are Instrumented **2**

3 NixNan Feature Reference **2**

4 Sampling, Level 1: Every Kernel Call **3**

5 Sampling, Level 2: Once in 8 Kernel Calls **4**

6 Sampling, Level 3: An Exponentially Decaying Schedule **4**

7 Histograms per Precision Bucket **5**

8 Making BF16 and FP32 Fail Too **5**
8.1 Where Infinity Actually Comes From 7

9 FP64: A Diagnostic Bug, and the Real Overflow **8**
9.1 The bug 8
9.2 Running FP64 for real, until it does overflow 9

10 Conclusion: What nixnan Made Visible Here **10**

1 The Example

`rd_nixnan.cu` solves a 1D reaction-diffusion PDE, $\partial u/\partial t = D\,\partial^2 u/\partial x^2 + \lambda u$, with explicit forward-Euler time stepping, on a 101-point grid for 1600 steps. The same step is implemented once per floating-point format — FP16, BF16, FP32, FP64 — as four separate CUDA kernels, so nixnan can attribute every exception and every exponent observation to a specific, known precision.

Why this example is precision-sensitive.

- The reaction term grows the solution by a fixed factor ($\sim$1.05$\times$) every step, so the exponent of every value climbs steadily regardless of format — the *only* thing that differs between formats is how much exponent range each one has before it overflows. FP16's 5-bit exponent runs out first, at step 300; BF16's, FP32's, and FP64's 8/8/11-bit exponents all still have headroom left at step 1600 (see Section 7).
- Because the kernel is otherwise identical across formats, any behavioral difference observed — overflow or not, subnormal or not, NaN cascade or not — is attributable entirely to the format's exponent width, not to any change in the algorithm. That makes it a clean instrument for isolating precision effects, and correspondingly a clean instrument for showing what nixnan actually measures.

2 "Straight nixnan": How These Runs Are Instrumented

Every run in this experiment uses `nixnan.so` directly, with no wrapper script:

```
LD_PRELOAD=<path>/nixnan.so [ENV_VARS...] ./rd_nixnan
```

`nixnan.so` is an NVBit tool: preloaded into the CUDA process, it rewrites each kernel's SASS the first time that kernel is launched, inserting device-side checks after (and before) every floating-point instruction. At exit it prints an aggregate report, per format, of unique exception *sites* (distinct instruction/operand locations that ever fired) and *repeats* (additional occurrences beyond each site's first hit) — the distinction that matters throughout this report.

3 NixNan Feature Reference

Before walking through results, this section fixes the exact meaning of every nixnan-specific setting the rest of the report relies on.

The `BIN_SPEC_FILE` JSON format. nixnan's own `--help` banner gives this example:

```
{
    "count": 128,
    "bf16": [],
    "f16": [[0,5],[-4,-1]],
    "f32": [],
    "f64": []
}
```

Each format key holds a list of *unbiased signed exponent* ranges to watch (here, only `f16` watches anything: the ranges $[0, 5]$ and $[-4, -1]$). `count` is a global report-frequency threshold: within a

watched range, nixnan counts every value landing in it and emits a "bin has reached threshold" line every `count` occurrences — so `count:128` here means "tell me every 128th time an `f16` value's exponent falls in $[0, 5]$ or $[-4, -1]$." Ranges are *unbiased* (e.g. FP16's usable exponent span is $[-14, 15]$, not the raw 5-bit field $[0, 31]$), and the two special encodings zero and infinity are structurally excluded from every range — a consequence used directly in Section 8.1: a value that jumps straight from finite to `inf` in one step, with no gradually-climbing trail through large-but-finite exponents first, is invisible to this mechanism no matter how the ranges are drawn. This experiment's own `HISTOGRAM=1` runs (Section 7) use the simpler, rangeless form of this same machinery: with no `BIN_SPEC_FILE` set at all, nixnan still tracks every exponent it sees and reports each format's overall $[\min, \max]$ at exit, just without the interim "every Nth occurrence" pings.

`SAMPLING=8`. Per nixnan's own banner text, *"Instrument a repeat kernel every SAMPLING times."* Concretely: the tool keeps a per-kernel launch counter, and only *enables* the already-instrumented version of the kernel on launches where `launch_count % SAMPLING == 0`; other launches run the kernel's plain, uninstrumented SASS. `SAMPLING=8` therefore observes 1 launch in 8; `SAMPLING` unset (or 0) disables sampling entirely, i.e. every launch is observed — the baseline used throughout Section 4.

How the exponential-decay parameters are specified. There is no `nixnan.so` setting for a decaying rate — `SAMPLING` is read once at process start and held fixed for that process's entire lifetime. The schedule in Section 6 is therefore specified entirely outside nixnan, in `shell-scripts/sampling_schedul` (three fields: `levels`, the sequence of `SAMPLING` values to sweep through; `snaps_per_level`, how many independent process launches to repeat at each level before advancing; and `cycles`, how many times to repeat the whole level sequence from the start). `03_sampling_exp_decay.sh` reads that file and, for every `(cycle, level, snap)` triple, launches exactly one fresh `SAMPLING=$level LD_PRELOAD=nixnan.so ./rd_nixnan` process, redirecting its full output to its own log file.

What generated `03_sampling_exp_decay_summary.tsv`. Every row of that file is the FP16 NaN/Infinity unique-site and repeat counts parsed back out of one such snap's log — 4 levels $\times$ 4 snaps $\times$ 2 cycles $= 32$ independent nixnan runs in total, each identical in every setting except `SAMPLING` and which `(cycle,level,snap)` log file it wrote to. No other nixnan setting (no `HISTOGRAM`, no `BIN_SPEC_FILE`, no `INSTR_MEM`) is active during these runs — it is the same default exception-instrumentation (`INSTRUMENT_EXCEPTIONS=1`) as Section 4's baseline, run 32 times at 4 different `SAMPLING` values.

4 Sampling, Level 1: Every Kernel Call

The baseline (`SAMPLING` unset, which nixnan treats as "instrument everything"):

```
LD_PRELOAD=nixnan.so ./rd_nixnan
```

Result (`Logs/01_sampling_every_call.log`), FP16 — the only format that overflows in this build:

```
NaN: 22 (60534 repeats)
Infinity: 22 (639 repeats)
```

This is the reference point every other sampling level in this report is compared against.

5 Sampling, Level 2: Once in 8 Kernel Calls

```
SAMPLING=8 LD_PRELOAD=nixnan.so ./rd_nixnan
```

Result (`Logs/02_sampling_every_8.log`):

```
NaN: 22 (7524 repeats)
Infinity: 17 (66 repeats)
```

Repeats drop by almost exactly 8× (60534/7524 ≈ 8.05), consistent with observing 1/8 of the kernel launches. But note the unique Infinity-site count also dropped, 22 → 17 — five distinct exception sites simply never got observed at this sampling rate. This is the first concrete evidence that `SAMPLING` trades completeness for overhead, not just verbosity for overhead.

A regression, now fixed. The pre-existing `run_nixnan_experiment.sh` in this directory carries a comment, from an earlier investigation, stating *"SAMPLING is a no-op in this nixnan build."* We re-tested that claim directly against the current build before writing this experiment (comparing `SAMPLING=1` against `SAMPLING=100000` on `reaction_diffusion_gpu_fp32`): at `SAMPLING=100000` every exception count dropped to exactly zero, which only happens if `SAMPLING` is genuinely gating instrumentation. `SAMPLING` is no longer a no-op in the `auto-nixnan` branch as built here — whatever caused the earlier no-op behavior has since been fixed.

6 Sampling, Level 3: An Exponentially Decaying Schedule

`nixnan.so` reads `SAMPLING` once at process start; there is no way to vary it mid-run. A decaying/backoff sampling rate therefore has to be built at the orchestration level, as a sequence of short runs ("snaps"), each holding `SAMPLING` constant. `shell-scripts/sampling_schedule.json` specifies that sequence (full rationale in its own header comment):

```
{
  "snaps_per_level": 4,
  "levels": [1, 2, 4, 8],
  "cycles": 2
}
```

i.e. step 4 times at spacing 1, double to spacing 2 and step 4 more times, double again to 4, double again to 8, then reset back to spacing 1 for a second cycle. `shell-scripts/03_sampling_exp_decay.sh` runs this schedule and tabulates the FP16 counts per snap (`Logs/03_sampling_exp_decay_summary.tsv`):

SAMPLING level	NaN unique	NaN repeats	Inf unique	Inf repeats
1	22	60534	22	639
2	22	30246	21	312
4	22	15103	17	154
8	22	7524	17	66

Every one of the 4 snaps at a given level is byte-identical (the program is deterministic and each snap is a fresh, independent process), and **cycle 2 is byte-identical to cycle 1 at every level** — the reset back to spacing 1 reproduces the exact baseline from Section 4, confirming there is no hidden cross-run state. See Figure 1 (`viz/sampling_decay.svg`).

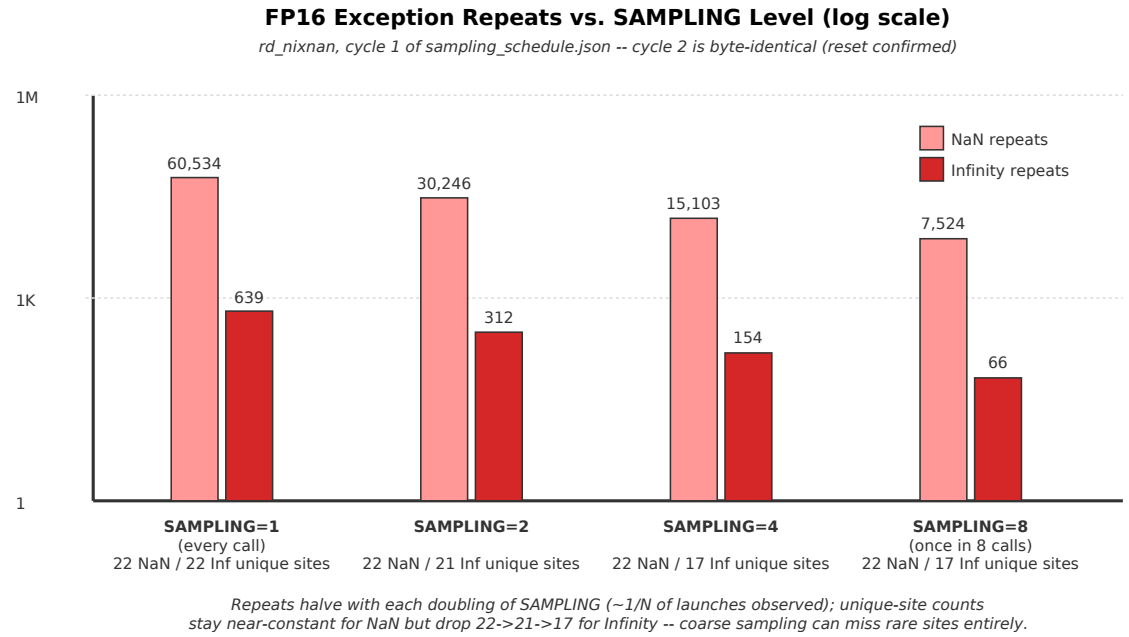


Figure 1: Repeats halve with each doubling of SAMPLING; unique-site counts stay near-constant for NaN but step down for Infinity.

7 Histograms per Precision Bucket

`HISTOGRAM=1` enables exponent-histogram tracking for every instrumented FP instruction, independent of the exception checker; at exit nixnan prints the [min, max] exponent actually observed for each format that had any instrumented activity:

```
HISTOGRAM=1 LD_PRELOAD=nixnan.so ./rd_nixnan
```

Result (`Logs/04_histograms_per_precision.log`):

```
Exponent range for bf16: [zero, 113]
Exponent range for f16: [zero, inf]
Exponent range for f32: [zero, 113]
Exponent range for f64: [zero, 113]
```

This is the direct, per-format confirmation of the precision-sensitivity claim from Section 1: FP16 is the only format whose exponent range is exhausted (it reports `inf` instead of a finite maximum) within 1600 steps. BF16, FP32, and FP64 all reach the identical exponent 113 — they are following the exact same growth trajectory (same algorithm, same input) and have not yet diverged from each other; only their exponent *ceilings* differ, and only FP16's has been reached.

8 Making BF16 and FP32 Fail Too

Every result so far has one format failing (FP16) and three staying finite. A natural follow-up: what would it take to make BF16 and FP32 fail as well, and does `nixnan` help answer that?

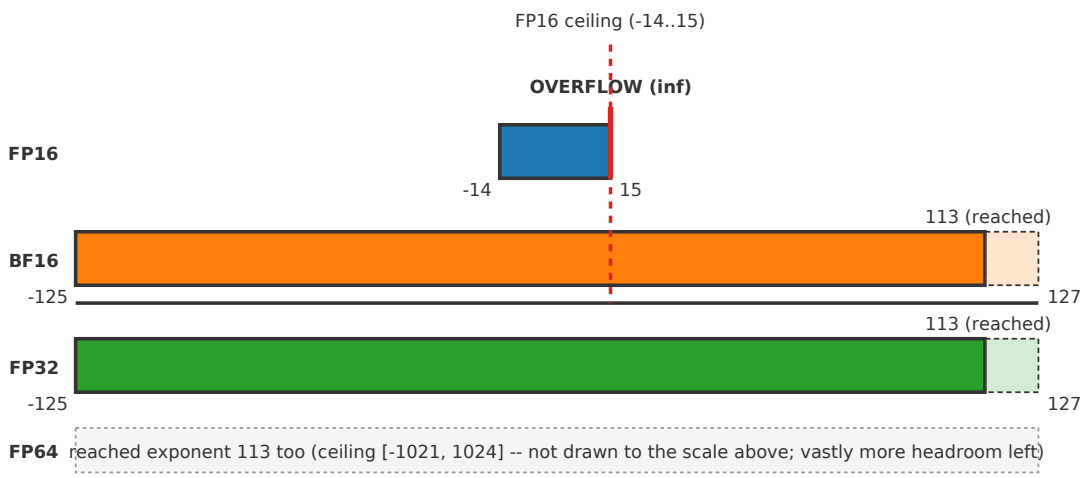


Figure 2: Exponent range actually reached by each format through step 1600. FP16's fixed capacity is $[-14, 15]$ and it exceeds it (overflow); BF16/FP32/FP64 all reach the same exponent, 113, with room remaining to their much wider ceilings.

Hypothesis tried: domain enlargement. Enlarging the grid (N, holding dx fixed by extending L proportionally) was tried *first*, because it is the most obvious "make the problem bigger" lever. It does not work, for a structural reason visible directly in the kernel: the reaction term `lam * u[i]` is a per-point multiplicative growth factor that does not reference N or dx at all — every grid point grows by the same $\sim 1.05\times$ per step regardless of how many points there are. Enlarging N just runs the identical per-step growth curve on more points; it cannot change *when* any single point's exponent is exhausted. This was already verified empirically earlier this session with `rd_nixnan_large.cu` (a $10\times$ grid, N=1001): at the same M=1600, only FP16 failed there too, exactly as at N=101.

What actually works: more time steps. `rd_nixnan.cu`'s step count is a compile-time macro (`#ifndef M ... #define M 1600`), so it can be overridden at compile time without touching the source:

```
nvcc -arch=sm_86 -DM=2000 rd_nixnan.cu -o rd_nixnan_extended
LD_PRELOAD=nixnan.so ./rd_nixnan_extended
```

`shell-scripts/05_extended_run_all_fail.sh` builds and runs exactly this (`Logs/05_extended_M2000.log`):

Format	First non-finite (step)	NaN unique/repeats	Inf unique/repeats
FP16	300	22 / 78134	22 / 639
BF16	1900	15 / 8975	6 / 76
FP32	1900	19 / 10966	8 / 90
FP64	— (correctly finite)	0 / 0	

At M=2000, BF16 and FP32 both now show real, nixnan-classified NaN/Infinity activity — confirmed, not just inferred from the program's own `max=...` printout. FP64 correctly stays finite at

M=2000: an earlier draft of this experiment reported FP64 also crossing to non-finite at step 1900, which turned out to be a bug in the driver's own check, not a real GPU event. Section 9 gives the fix and then runs FP64 for long enough that it does overflow for real.

8.1 Where Infinity Actually Comes From

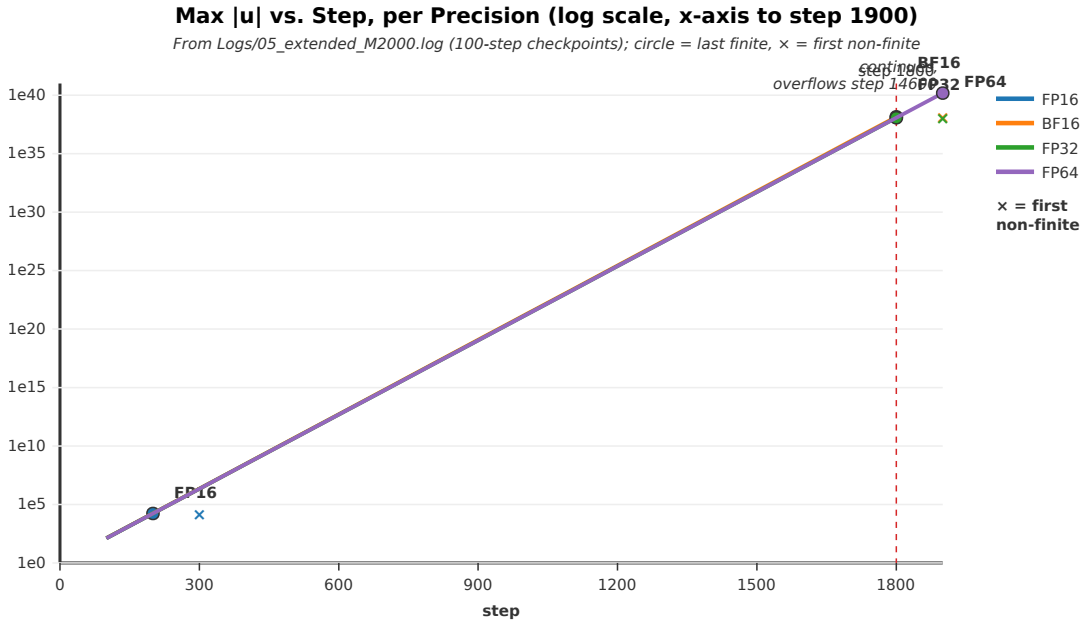


Figure 3: Max$|u|$ per 100-step checkpoint, all four formats, from the same `Logs/05_extended_M2000.log` run as the table above (regenerated after the Section 9 fix). FP16 (blue) departs the shared curve at step 300. BF16/FP32/FP64 stay numerically indistinguishable all the way to step 1800 (1.1675×10^{38} for FP32 and FP64 to all four significant figures shown); BF16 and FP32 are non-finite by step 1900, while FP64 continues (its real value there is $\approx 1.5 \times 10^{40}$ — large, but still finite).

The CUDA statement where Infinity was first generated. `rd_nixnan.cu` was rebuilt with `-g -lineinfo` for this run specifically so nixnan's `LINE_INFO=1` (the default) could resolve real source locations instead of SASS only. For FP32, it does, cleanly:

```
error [infinity] detected in operand 0 of instruction FADD R0, R0, R7
  ; in function rd_step_fp32 at rd_nixnan.cu:96 of type f32
```

Line 96 is the Laplacian: `float lap = (u[i + 1] + u[i - 1]) - 2.f * u[i];` — **Infinity first appears while computing the diffusion term, not the reaction term.** BF16 partially resolves too (its analogous line, 84, appears on some but not all of its error lines — likely because BF16's arithmetic goes through `__h*` intrinsic wrappers that are not always attributed as cleanly as FP32's plain operators). FP16's error lines carry no source location at all: every FP16 statement uses `cuda_fp16.h` intrinsics (`__hadd`, `__hsub`, `__hmul`), which are fully inlined and, in this build, resolve to no attributable line whatsoever — a real limitation of `nixnan`'s line-info resolution for deeply-inlined intrinsics, not a property of the kernel. By direct structural analogy (line 70,

__hsub(__hadd(u[i+1],u[i-1]), __hmul(two,u[i])), is the exact same expression as FP32's line 96, and FP16's SASS shows the identical HADD2-then-HFMA2-with-constant-2 pattern nixnan reports for FP32/BF16 at their resolved lines), the same conclusion almost certainly holds for FP16, though nixnan itself cannot confirm it here.

How Infinity becomes NaN, with the specific statement. The original source comment in rd_nixnan.cu predicted this mechanism by hand: inf - 2*inf + inf -> NaN. nixnan's own instrumentation confirms it, to the instruction, for FP32. The three FADD instructions on line 96 fire in this order as the classifier first sees each:

```
FADD R0, R0, R7 ; u[i+1] + u[i-1] -> operand already Infinity
FADD R7, R6, R6 ; 2.f * u[i] (as u[i]+u[i]) -> operand already Infinity
FADD R7, R0, -R7 ; (u[i+1]+u[i-1]) - 2.f*u[i] -> RESULT is NaN
```

The third instruction is exactly $\infty - \infty$, IEEE-754's indeterminate form — nixnan reports NaN for that instruction's *result* operand as the very next distinct site to fire. That NaN then flows into line 97's FFMA chain (un[i] = u[i] + r * lap + lam * u[i];) and out through the final STG.E store, exactly matching the source comment's prediction and giving it a concrete, tool-verified instruction sequence rather than a hand-derived guess.

9 FP64: A Diagnostic Bug, and the Real Overflow

Section 8 reported FP64 crossing to non-finite at step 1900. That report was wrong — a bug in the program's own check, described below and now fixed — and FP64 does eventually overflow for real, just much later (Section 9.2).

9.1 The bug

rd_nixnan.cu converts every format's grid values to double for the non-finite check, via a small helper:

```
template <class T> static double from_T(T);
...
template <> double from_T<double>(double v){ return v; }
```

from_T<double> returns the value unchanged. The check that calls it, however, assigned the result into a float:

```
float mx = -1e30f; bool bad = false;
for (int i = 0; i < N; ++i) {
    float v = from_T<T>(h[i]); // <-- narrows a double into a float
    if (!std::isfinite(v)) bad = true;
    if (std::isfinite(v) && v > mx) mx = v;
}
```

float's maximum finite magnitude is $\approx 3.4028235 \times 10^{38}$. Once a double value exceeds that, narrowing it into a float produces +Infinity by itself — regardless of whether the double was ever non-finite. At step 1900, FP64's real values are of order 1.5×10^{40}: past float's ceiling, but $\sim$270 orders of magnitude below double's actual ceiling ($\sim 1.8 \times 10^{308}$), and perfectly finite. That one narrowing conversion is the entire cause of the step-1900 report; there was no GPU exception for nixnan to have caught, and its zero was correct.

The fix. Keep the check in `double` throughout (`rd_nixnan.cu`, committed separately, commit `03c9ef3`):

```
double mx = -1e30; bool bad = false;
for (int i = 0; i < N; ++i) {
    double v = from_T<T>(h[i]);
    if (!std::isfinite(v)) bad = true;
    if (std::isfinite(v) && v > mx) mx = v;
}
```

The rest of the file was audited for the same class of mistake: every other `float/half/bf16` cast (in `to_T/from_T` and the kernels themselves) is an intentional, correctly-directed conversion to a type's own native precision, not a truncation of a value already under test. No other instance was found. With the fix in place, re-running Section 8's M=2000 build (Figure 3, updated table) shows FP64 correctly staying finite, with `nixnan` reporting 0/0 for it, as it always should have.

9.2 Running FP64 for real, until it does overflow

FP64's actual ceiling is $\sim 1.8 \times 10^{308}$. From its 1.1675×10^{38} value at step 1800 and the observed $\sim$1.05×/step growth, reaching that ceiling takes on the order of $\log_{10}(1.8 \times 10^{308}/1.1675 \times 10^{38})/\log_{10}(1.05) \approx 12{,}750$ more steps — an overflow step around 14,550.

`rd_nixnan.cu` gained a build mode, `FP64_LONGRUN` (off by default, so every earlier stage in this experiment is unaffected), to actually reach that point:

```
nvcc -arch=sm_86 -g -lineinfo -DFP64_LONGRUN=1 rd_nixnan.cu -o rd_nixnan_fp64_longrun
LD_PRELOAD=nixnan.so ./rd_nixnan_fp64_longrun
```

It runs all four precisions together up to `SUPPRESS_AFTER` (3800 steps, twice FP32's real trigger step of 1900), then frees FP16/BF16/FP32's buffers — already long non-finite by then — and steps FP64 alone up to `M_FP64_MAX` (20000) or first non-finite. `shell-scripts/06_fp64_longrun.sh` builds and runs this (`Logs/06_fp64_longrun.log`). FP64 overflows at step 14600 (t=14.6), matching the estimate above, and `nixnan` now classifies it for real:

	NaN (unique/repeats)	Infinity (unique/repeats)	−Infinity (unique/repeats)
FP64 (this run)	36 / 1699	10 / 74	32 / 712

With `-lineinfo`, the first Infinity is attributed to the same statement as FP32's, `rd_nixnan.cu:106`, the Laplacian (`double lap = (u[i + 1] + u[i - 1]) - 2.0 * u[i];`):

```
error [infinity] detected in operand 0 of instruction DADD R6, R6, R8
  ; in function rd_step_fp64 at rd_nixnan.cu:106 of type f64
```

and the same three-instruction pattern turns it into `NaN`:

```
DADD R6, R6, R8 ; u[i+1] + u[i-1] -> operand already Infinity
DADD R8, R10, R10 ; 2.0 * u[i] (as u[i]+u[i]) -> operand already Infinity
DADD R6, R6, -R8 ; (sum) - 2.0*u[i] -> RESULT is NaN
```

That `NaN` then flows into line 107's `DFMA` chain and out through the final `STG.E.64` store — the identical mechanism as FP32 (Section 8.1), at a much larger step count because FP64 simply has far more exponent range to climb through first.

10 Conclusion: What nixnan Made Visible Here

Across six short experiments on one simple, well-understood kernel, `nixnan` surfaced several things that would not be visible from the program's own output (a plain run of `rd_nixnan` only prints `max=...` per step, with no notion of exception sites, repeats, or exponent range):

1. **Sampling is a real overhead/completeness knob, not a free lunch.** Coarser sampling doesn't just reduce reported repeats proportionally — it can silently drop entire exception sites from the report (22 → 17 unique Infinity sites, Section 4), which matters if the goal is finding every distinct bug location rather than just gauging overall severity.
2. **A fixed env var can still drive an arbitrary sampling schedule**, by scripting a sequence of short, independent runs around it (Section 6) — and because `rd_nixnan` is fully deterministic, that schedule is provably reproducible: cycle 2 reproduces cycle 1 exactly.
3. **Per-format exponent histograms directly confirm *which* format is precision-sensitive and by how much** (Section 7), turning "FP16 fails and the others probably don't" into a specific, quantified claim: FP16 alone hits `inf`; the other three all reach exactly the same exponent, 113, with headroom still remaining to their much larger ceilings.
4. **nixnan can adjudicate a hypothesis about *why* a format fails first** (Section 8): domain enlargement, the obvious first guess, is ruled out directly by the kernel's own growth term; running longer is confirmed, with nixnan's own classifier newly reporting real BF16 and FP32 exceptions at M=2000 where it reported none at M=1600.
5. **Line-level attribution turns a hand-derived guess into a tool-verified fact** (Section 8.1): with `-lineinfo`, nixnan pinpoints the exact CUDA statement (the Laplacian, not the reaction term) and the exact SASS instruction where $\infty - \infty$ produces `NaN` — confirming, to the instruction, a mechanism the source code's own comments could only predict by hand.
6. **FP64 does eventually fail too, and nixnan shows exactly where** (Section 9): an implicit `double`→`float` narrowing in the program's own check had falsely reported FP64 failing at step 1900; fixed, then run for real to step 14600 (the `FP64_LONGRUN` build mode), FP64 overflows at the step its own growth rate predicts, and nixnan attributes it to the identical Laplacian statement and $\infty - \infty \rightarrow$ `NaN` mechanism already confirmed for FP32.

As a side effect, re-verifying `SAMPLING` against the current build (Section 4) also caught that a previously-documented no-op regression in `nixnan.so` has since been fixed — itself an example of nixnan-adjacent tooling needing the same kind of empirical re-verification its own instrumentation provides for user code.